

Successful CHyLL v2 Examples: User Guide

Data, models, and how to regenerate the downstream inputs

Kaito Iwasaki

Thursday walkthrough

Purpose of the Meeting

- Give David and Bernardo a clean bundle with only successful examples.
- Map each file to the mathematical/computational object it contains.
- Show how to load the data immediately.
- Explain what the CHyLL v2 losses enforce and where they live in code.
- Show how to regenerate the exported lift data from the trained models.
- Identify what David needs for his downstream computations.

What We Are Sending

The file to send is:

`2026-05-19_successful_examples_chyll_v2_repro.zip`

It contains:

- exported data for all successful examples,
- trained model weights and configs,
- minimal source code needed to reload/export,
- rollout figures for every successful model,
- training loss-history figures for every successful training,
- selected additional orientation figures,
- a README, examples table, requirements file, and hash manifest.

What Is Deliberately Not Included

The bundle excludes:

- smoke-test runs,
- failed compass uniform-initial-condition run,
- intermediate ablations,
- obsolete top-level `data/compass_gait/` files,
- older non-`chyll_v2` architecture outputs.

The goal is to avoid making David sort through bad or superseded data.

The Successful Examples

Folder	System	Meaning
rimless_wheel_phaseB	Rimless wheel	Stable limit cycle
bouncing_ball_phaseB_5impacts	Bouncing ball	5-impact lift
bouncing_ball_phaseB_15impacts	Bouncing ball	15-impact lift
compass_period2_4p75deg_phi1	Compass gait	4.75 deg, period 2
compass_period4_5p00deg_phi2	Compass gait	5.00 deg, period 4
compass_period8_5p02deg_phi3	Compass gait	5.02 deg, period 8
compass_chaotic_cloud_5p20deg_phi4	Compass gait	5.20 deg, chaotic/cloud

One-Sentence Big Picture

Each example is a hybrid dynamical system trajectory embedded by CHyLL v2 into a continuous latent space.

The exported files give David:

- points along the latent trajectory,
- tangent directions along that trajectory,
- the trained model that generated them,
- and the existing cycling-signature output files.

Notation and Artifact Map

Artifact	Meaning
<code>data/<case>/</code>	exported lifts and downstream outputs
<code>models/<case>/</code>	trained CHyLL v2 checkpoint and run config
<code>latent_lift_positions.npy</code>	sampled points in learned latent space
<code>latent_lift_tangents.npy</code>	tangent vectors along the sampled lift
<code>postimpact_*</code>	compass Poincare-section data after impact
<code>cycling_signature_outputs/</code>	existing downstream outputs for comparison

Convention: `<case>` denotes one row of `examples.csv`.

Commands Used in the Demo

The only commands needed for the walkthrough are:

```
cd <bundle folder>
python -m pip install -r requirements.txt
python export_all_lifts.py
```

Replace <bundle folder> by the unzipped bundle path. If the path contains spaces, put quotes around it.

Top-Level Folder Layout

After unzipping:

<code>data/</code>	ready-to-use exported data
<code>models/</code>	trained model weights and configs
<code>source/</code>	source code needed to reload/export
<code>figures/</code>	rollout, loss-history, and orientation plots
<code>README.md</code>	text user guide
<code>examples.csv</code>	table of examples
<code>export_all_lifts.py</code>	regenerate exported lifts
<code>requirements.txt</code>	Python package list
<code>MANIFEST.csv</code>	file hashes

Rollout and Loss Figures

The `figures/` folder includes every successful rollout overlay:

- rimless wheel: short, index, and long rollout plots,
- bouncing ball: standard and index rollout plots,
- compass gait: one rollout plot for each incline case,
- loss-history plots for the six successful trainings,
- plus rank heatmaps and the compass latent-cluster panel.

These are diagnostic plots. The arrays for downstream computation are still the `data/<case>/latent_lift_*` files.

Data Folder Anatomy

Each example folder in data/ contains:

- `latent_lift_positions.npy`
- `latent_lift_tangents.npy`
- `latent_lift_positions.csv`
- `latent_lift_tangents.csv`
- `export_report.txt`
- `cycling_signature_outputs/`

The `.npy` files are best for Python. The `.csv` files are easier for spreadsheets and quick inspection.

What Are Positions and Tangents?

- `latent_lift_positions` is the sequence of points in latent space.
- `latent_lift_tangents` is the direction of motion at each point.
- The two arrays have the same shape.
- David's cycling-signature code uses both positions and tangents.

Example: a shape (960, 7) means:

- 960 sampled points,
- each point has 7 latent coordinates.

Shapes of the Exported Lifts

Example	Position/tangent shape
Rimless wheel Phase-B	(960, 7)
Bouncing ball, 5 impacts	(1069, 7)
Bouncing ball, 15 impacts	(2004, 7)
Compass 4.75 deg, period 2	(2032, 11)
Compass 5.00 deg, period 4	(2036, 11)
Compass 5.02 deg, period 8	(2036, 11)
Compass 5.20 deg, chaotic/cloud	(2031, 11)

Compass-Specific Files

The compass-gait folders also include:

- `postimpact_states.npy`: physical post-impact states.
- `postimpact_latent_z.npy`: encoded post-impact section points.
- `postimpact_labels.npy`: cluster labels for the period diagnostic.

These files support the incline-cascade interpretation: period 2, period 4, period 8, and chaotic/cloud.

Model Folder Anatomy

Each folder in `models/` contains:

- `model.pt`: learned neural-network weights.
- `config.json`: model/system settings used for that run.
- `train_log.jsonl`: training log.
- optional `console.*.log`: raw console output for some runs.

The data can be used without loading the models. The models are included so David can regenerate the data or modify export settings.

Learned Objects

CHyLL v2 learns three maps on the augmented mapping-cylinder state $y = (x, s) \in X'$:

$$E_\theta : X' \rightarrow \mathbb{R}^m$$

encoder,

$$V_\theta : \mathbb{R}^m \rightarrow \mathbb{R}^m$$

latent vector field,

$$D_\theta : \mathbb{R}^m \rightarrow X'$$

decoder.

Here $m = 7$ for rimless/bouncing and $m = 11$ for compass gait.

Implementation Map

Mathematical object	Code location
$E_\theta, V_\theta, D_\theta$	chyll_v2/networks.py
latent ODE rollout	chyll_v2/ode.py
loss terms L_x, L_z, L_g, L_v, L_c	chyll_v2/losses.py
training loop / checkpoint saving	chyll_v2/train.py
system simulators and reset maps	chyll_v2/systems/
export scripts	cycling_signature/export/

All paths here are relative to source/chyll_v2/.

Forward Pass During Training

For a sampled trajectory slice $y_0, \dots, y_{T-1}$:

$$\begin{aligned} z_i &= E_\theta(y_i), \\ \hat{z}_i &= \Psi_\theta(t_i; z_0), \quad \dot{z} = V_\theta(z), \\ \hat{y}_i &= D_\theta(\hat{z}_i). \end{aligned}$$

In code this is `forward_step(...)` in `train.py`: encode the observed slice, integrate the Neural ODE, decode the integrated latent trajectory, then call `compute_losses(...)`.

Composite Loss

The optimized objective is:

$$L = w_x L_x + w_z L_z + w_g L_g + w_v L_v + w_c L_c.$$

Included successful examples use:

- rimless/bouncing: $(w_x, w_z, w_g, w_v, w_c) = (1, 1, 3, 1, 1)$,
- compass: $(w_x, w_z, w_g, w_v, w_c) = (1, 1, 3, 0, 1)$.

The exact values are in each `models/<case>/config.json`.

What the Losses Enforce

Term	Mathematical role	Code helper
L_x	$D_\theta(\widehat{z}_i) \approx y_i$ reconstruction	<code>reconstruction_loss</code>
L_z	$\Psi_\theta(t_i, E_\theta(y_0)) \approx E_\theta(y_i)$	<code>latent_consistency_loss</code>
L_g	contract reset/gluing pairs in latent space	<code>gluing_loss</code>
L_v	match pre/post seam velocities where enabled	<code>seam_velocity_loss</code>
L_c	keep latent coordinates from collapsing	<code>anti_collapse_loss</code>

The glue/seam masks are data-driven: they are inferred from the sampled mapping-cylinder coordinate s , not from symbolic guard equations.

How the Learned Models Are Found

- Training writes one run directory per successful case.
- `model.pt` is the PyTorch `state_dict`.
- `config.json` reconstructs dimensions, system parameters, loss weights, and ODE/training settings.
- `train_log.jsonl` records L_x, L_z, L_g, L_v, L_c over training.

The handoff copies only the successful checkpoints into `models/`.

How the Export Script Uses a Model

Each entry in `export_all_lifts.py` specifies:

```
"model": ROOT / "models" / <case> / "model.pt"  
"config": ROOT / "models" / <case> / "config.json"  
"script": ... / "prepare*_lift.py"
```

The export script rebuilds the network from `config.json`, loads `model.pt`, simulates the corresponding hybrid example, encodes it with E_θ , computes tangents, and writes `regenerated/<case>/`.

Three Levels of Use

- 1 **Use existing data.** Load `latent_lift_positions.npy` and `latent_lift_tangents.npy`.
- 2 **Regenerate lift data.** Run `python export_all_lifts.py`.
- 3 **Inspect or modify model-derived exports.** Edit only export parameters first; retraining is not needed for Thursday.

For Thursday, we should demo levels 1 and 2 and show where the model files enter the regeneration step.

Loading Data in Python

Minimal example:

```
import numpy as np

case = "compass_period4_5p00deg_phi2"
Z = np.load(f"data/{case}/latent_lift_positions.npy")
T = np.load(f"data/{case}/latent_lift_tangents.npy")

print(Z.shape)
print(T.shape)
```

Here Z is the latent position array and T is the tangent array.

Loading Compass Post-Impact Data

For compass-gait period/cascade analysis:

```
states = np.load(f"data/{case}/postimpact_states.npy")
section_z = np.load(f"data/{case}/postimpact_latent_z.npy")
labels = np.load(f"data/{case}/postimpact_labels.npy")

print(states.shape)
print(section_z.shape)
print(labels.shape)
```

The labels are the cluster IDs used to count period-2, period-4, period-8, or many/cloud behavior.

Loading CSV Files Instead

If someone prefers CSV:

```
import numpy as np

Z = np.loadtxt("data/rimless_wheel_phaseB/latent_lift_positions.csv")
T = np.loadtxt("data/rimless_wheel_phaseB/latent_lift_tangents.csv")

print(Z.shape)
```

The CSV files are space-delimited; use a space delimiter in any non-Python tool.

What the Export Script Does

The script:

- loads each bundled `model.pt`,
- reads the matching `config.json`,
- simulates the appropriate hybrid trajectory,
- encodes it into CHyLL v2 latent space,
- computes tangent directions,
- writes outputs to `regenerated/<case>/`.

It does not retrain models. It only regenerates exported data.

Python Setup

From the unzipped bundle folder:

```
python -m pip install -r requirements.txt
```

Then:

```
python export_all_lifts.py
```

This should create:

```
regenerated/rimless_wheel_phaseB/  
regenerated/bouncing_ball_phaseB_5impacts/  
...
```

The Role of `source/`

`source/` contains the minimal CHyLL v2 code needed to reload models:

- network architecture,
- system simulators,
- lift exporters,
- downstream helper scripts.

It is included so David does not need to reconstruct the repo structure from scratch just to regenerate the data.

Rimless and Bouncing-Ball Tangents

For rimless wheel and bouncing ball:

- exported tangents use tag-aware finite differences,
- this differentiates within each arc/bridge piece,
- it avoids artificial chords across concatenation boundaries.

In the export script this corresponds to:

```
-tangent-source diff -tangent-mode tagaware
```

Compass Tangents

For the compass incline cascade:

- exported tangents use the learned latent vector field,
- this matches the included files,
- it is the tangent convention used for the current compass cascade data.

In the export script this corresponds to:

```
-tangent-source vfield
```

Downstream Computation Boundary

Existing outputs are already included under:

```
data/<case>/cycling_signature_outputs/
```

- The handoff responsibility is to provide clean positions, tangents, and labels.
- David can plug those arrays into his existing downstream workflow.
- We do not need to explain or debug his downstream environment in the main walkthrough.

Common Mistakes

- Running commands from the wrong folder.
- Forgetting quotes around paths with spaces.
- Mixing old `data/compass_gait` files with CHyLL v2 files.
- Using the wrong model with the wrong config.
- Expecting `export_all_lifts.py` to train models.

Sanity Checks

After loading data:

- positions and tangents should have the same shape,
- tangent norms should be close to 1,
- compass post-impact states should be (256, 4),
- compass post-impact latent points should be (256, 11).

These checks are already done for the shipped bundle.

Tiny Python Sanity Check

```
import numpy as np

case = "rimless_wheel_phaseB"
Z = np.load(f"data/{case}/latent_lift_positions.npy")
T = np.load(f"data/{case}/latent_lift_tangents.npy")

norms = np.linalg.norm(T, axis=1)
print(Z.shape, T.shape)
print(norms.min(), norms.max())
```

The min and max should both print very close to 1.

Suggested Thursday Agenda

- 1 Confirm everyone is using the successful-examples zip.
- 2 Walk through `examples.csv` and folder layout.
- 3 Load one `.npy` data file in Python.
- 4 Explain the learned maps and loss terms at the level of the code.
- 5 Regenerate one example using `export_all_lifts.py`.
- 6 Explain the cycling-signature output folders.

Live Demo Plan

Recommended demo order:

- 1 Open the zip folder.
- 2 Open README.md.
- 3 Open examples.csv.
- 4 Load `data/compass_period4_5p00deg_phi2/latent_lift_positions.npy`.
- 5 Run `python export_all_lifts.py`.
- 6 Compare `data/` and `regenerated/`.

What David Can Safely Modify

- Which example/case to process.
- Cycling-signature parameters.
- Number and length of random subsegments.
- Export destination folder.
- Whether to use existing data or regenerated data.

He should not need to retrain models for the immediate downstream task.

Best Follow-Up Deliverable

After the meeting, it would be useful to send:

- the final zip,
- this slide deck,
- a one-paragraph note saying which examples are successful,
- and the exact command David should run first.

First command:

```
python export_all_lifts.py
```

Summary

- The zip is a clean successful-examples bundle.
- Data can be used immediately from `data/`.
- Models and source are included for regeneration.
- `export_all_lifts.py` is the easiest entry point.
- Downstream outputs are included for comparison; David can rerun them in his own workflow.